

NAME - AKSHAY CHAUHAN

SECTION- C

ROLL NO- 6

CUID - CU24250262

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
from google.colab import files

uploaded = files.upload()
```

Choose Files | car.jpg
car.jpg(image/jpeg) - 243110 bytes, last modified: 9/15/2026 - 100% done
Saving car.jpg to car (1).jpg

```
image_path = list(uploaded.keys())[0]

image = cv2.imread(image_path)

# Convert BGR to RGB for displaying with matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Original Image



```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(10, 6))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```

Grayscale Image



```
# Improve contrast using histogram equalization
gray_processed = cv2.equalizeHist(gray)
```

```
plt.figure(figsize=(10, 6))
plt.imshow(gray_processed, cmap="gray")
plt.title("Preprocessed Grayscale Image")
plt.axis("off")
plt.show()
```

Preprocessed Grayscale Image



```
# Create SIFT detector
sift = cv2.SIFT_create()

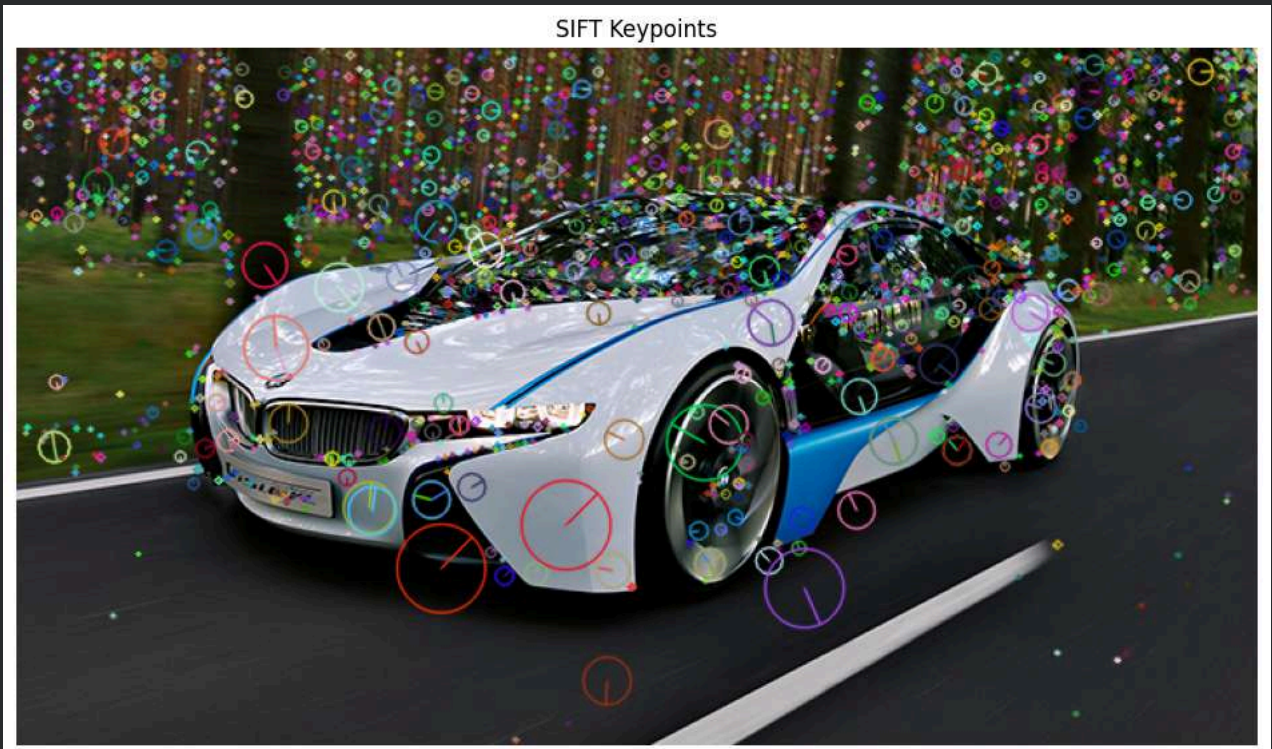
# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(gray_processed, None)
```

```
print("Number of keypoints detected:", len(keypoints))
print("Descriptor shape:", descriptors.shape if descriptors is not None else None)
```

```
Number of keypoints detected: 2382
Descriptor shape: (2382, 128)
```

```
sift_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
```

```
plt.figure(figsize=(12, 8))
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```



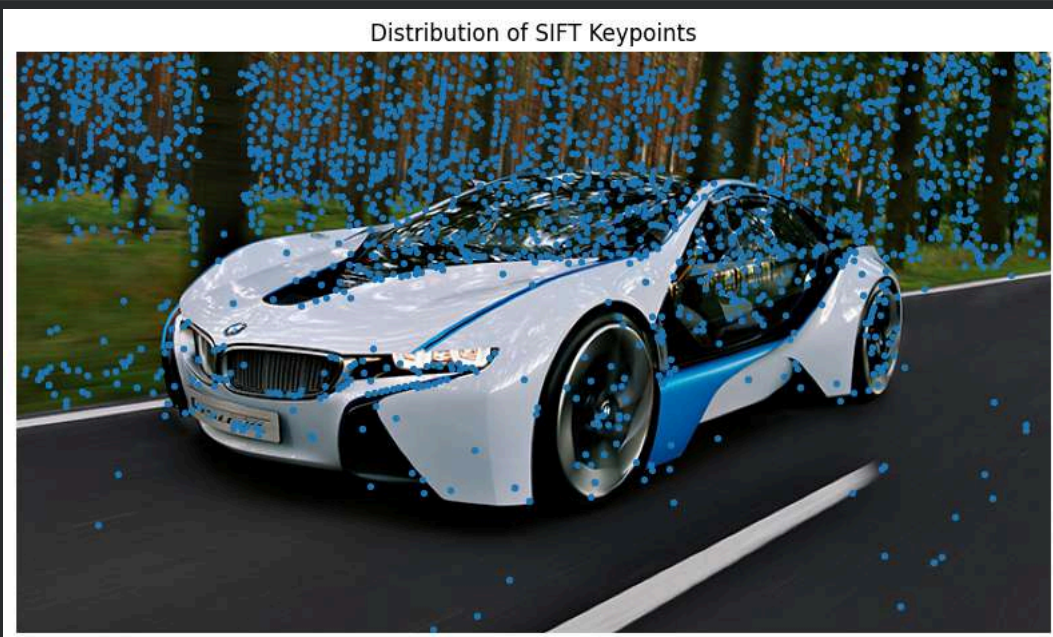
```
plt.figure(figsize=(10, 6))

plt.imshow(image_rgb)

x = [kp.pt[0] for kp in keypoints]
y = [kp.pt[1] for kp in keypoints]

plt.scatter(x, y, s=8)

plt.title("Distribution of SIFT Keypoints")
plt.axis("off")
plt.show()
```



```
hog_image = cv2.resize(gray_processed, (128, 128))

print("Image size for HOG:", hog_image.shape)

Image size for HOG: (128, 128)
```

```

features, hog_visualization = hog(
    hog_image,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys',
    visualize=True
)

print("Number of HOG features:", len(features))

```

Number of HOG features: 8100

```

# Rescale HOG visualization for better display
hog_visualization_rescaled = exposure.rescale_intensity(
    hog_visualization,
    in_range=(0, 10)
)

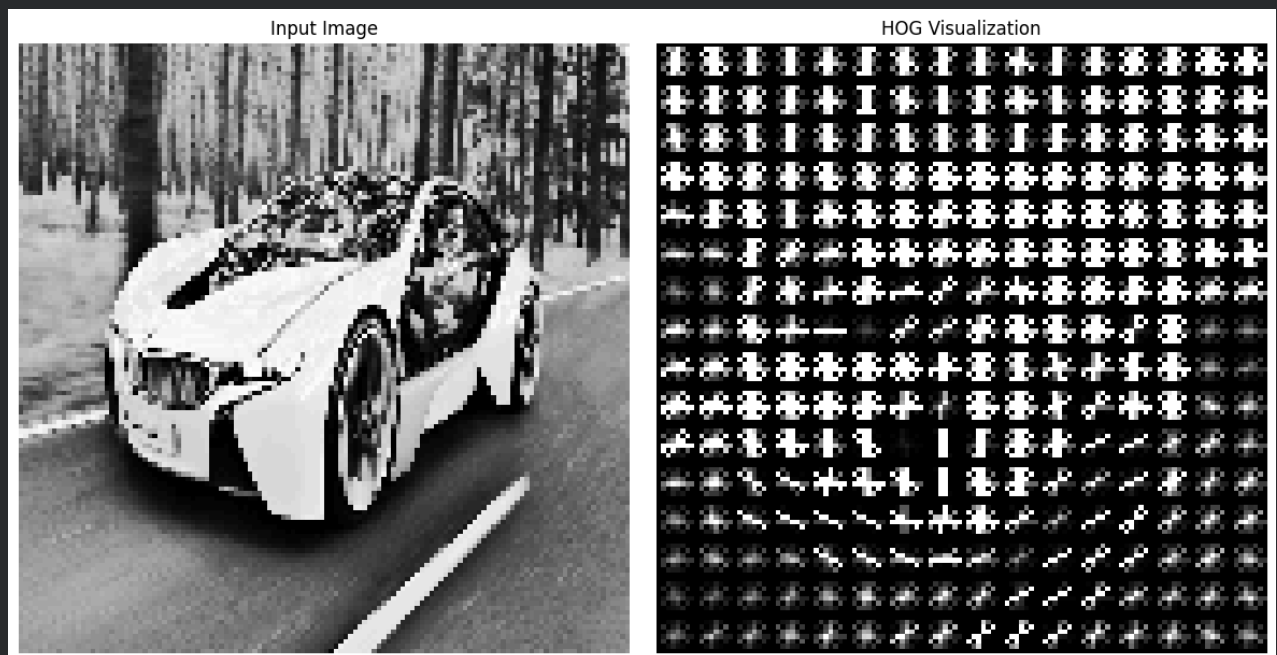
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(hog_image, cmap="gray")
plt.title("Input Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_visualization_rescaled, cmap="gray")
plt.title("HOG Visualization")
plt.axis("off")

plt.tight_layout()
plt.show()

```



```

print("HOG Parameters")
print("-----")
print("Orientations      :", 9)
print("Pixels per cell    :", (8, 8))
print("Cells per block     :", (2, 2))
print("Block normalization: L2-Hys")
print("Number of features:", len(features))

```

```

HOG Parameters
-----
Orientations      : 9
Pixels per cell   : (8, 8)
Cells per block   : (2, 2)
Block normalization: L2-Hys
Number of features: 8100

```

```

print("SIFT vs HOG")
print("=" * 50)

print("\nSIFT")
print("- Detects local keypoints")
print("- Produces local descriptors")
print("- Strong scale robustness")
print("- Provides rotation robustness")
print("- Useful for image matching and object recognition")

print("\nHOG")
print("- Describes local gradient orientations")
print("- Captures shape and edge information")
print("- Commonly used for object detection")
print("- Computationally simpler than many local feature matching pipelines")
print("- Not inherently scale and rotation invariant like SIFT")

```

SIFT vs HOG

=====

SIFT

- Detects local keypoints
- Produces local descriptors
- Strong scale robustness
- Provides rotation robustness
- Useful for image matching and object recognition

HOG

- Describes local gradient orientations
- Captures shape and edge information
- Commonly used for object detection
- Computationally simpler than many local feature matching pipelines
- Not inherently scale and rotation invariant like SIFT

```
uploaded2 = files.upload()
```

Choose Files | car2.jpg

car2.jpg(image/jpeg) - 81530 bytes, last modified: 9/15/2026 - 100% done
 Saving car2.jpg to car2.jpg

```

image2_path = list(uploaded2.keys())[0]

image2 = cv2.imread(image2_path)

gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

image2_rgb = cv2.cvtColor(image2, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 6))
plt.imshow(image2_rgb)
plt.title("Second Image")
plt.axis("off")
plt.show()

```

Second Image



```
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)
```

```
print("Keypoints in Image 1:", len(keypoints))
print("Keypoints in Image 2:", len(keypoints2))
```

```
Keypoints in Image 1: 2382
Keypoints in Image 2: 1148
```

```
# FLANN parameters
index_params = dict(
    algorithm=1,
    trees=5
)

search_params = dict(
    checks=50
)

flann = cv2.FlannBasedMatcher(
    index_params,
    search_params
)

matches = flann.knnMatch(
    descriptors,
    descriptors2,
    k=2
)

print("Total candidate matches:", len(matches))
```

```
Total candidate matches: 2382
```

```
good_matches = []

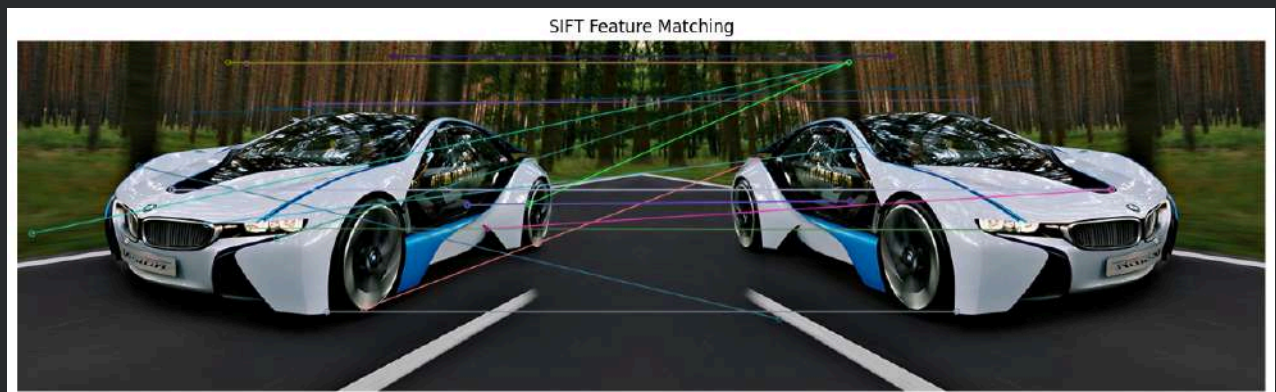
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

print("Good matches:", len(good_matches))
```

```
Good matches: 20
```

```
match_image = cv2.drawMatches(
    image_rgb,
    keypoints,
    image2_rgb,
    keypoints2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 8))
plt.imshow(match_image)
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()
```



```
print("OBSERVATIONS")
print("=" * 60)

print("1. SIFT successfully detected distinctive keypoints in the image.")
print("2. SIFT descriptors represented local image features using 128-dimensional descriptors.")
print("3. Keypoints were mainly concentrated around corners, edges and textured regions.")
```

```
print("4. HOG captured gradient orientation and shape information from the image.")
print("5. SIFT is more suitable for local feature matching and recognition.")
print("6. HOG is useful for representing object shape and gradient structure.")
print("7. SIFT provides strong robustness to scale and rotation changes.")
print("8. HOG is computationally useful for object detection but is not inherently")
print("    invariant to scale and rotation.")
print("9. SIFT matching successfully identified corresponding features between")
print("    the two similar images.")
```

OBSERVATIONS

- ```
=====
```
1. SIFT successfully detected distinctive keypoints in the image.
  2. SIFT descriptors represented local image features using 128-dimensional descriptors.
  3. Keypoints were mainly concentrated around corners, edges and textured regions.
  4. HOG captured gradient orientation and shape information from the image.
  5. SIFT is more suitable for local feature matching and recognition.
  6. HOG is useful for representing object shape and gradient structure.
  7. SIFT provides strong robustness to scale and rotation changes.
  8. HOG is computationally useful for object detection but is not inherently invariant to scale and rotation.
  9. SIFT matching successfully identified corresponding features between the two similar images.

## THEORY QUESTIONS AND ANSWERS

### Q1. What is feature extraction, and why is it important in computer vision?

#### Answer:

Feature extraction is the process of identifying and representing important characteristics or patterns present in an image. Instead of using all the raw pixel values, feature extraction converts an image into a set of meaningful features or descriptors.

Feature extraction is important in computer vision because these features can be used for **image classification, object recognition, image matching, object tracking, and detection**. It reduces the complexity of image data and helps computer vision systems identify useful visual information such as edges, corners, shapes, textures, and local patterns.

A good feature extraction technique should provide features that are robust to changes such as **scale, rotation, illumination, and viewpoint**.

### Q2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

#### Answer:

SIFT stands for **Scale-Invariant Feature Transform**. It is a feature extraction technique used to detect distinctive points in an image and generate descriptors for those points.

The main steps of SIFT are:

1. **Scale-space construction:** Different scaled versions of the image are created to detect features at different sizes.
2. **Keypoint detection:** The Difference of Gaussian (DoG) method is used to identify potential keypoints.
3. **Keypoint localization:** Unstable or low-contrast points are removed and stable keypoints are selected.
4. **Orientation assignment:** An orientation is assigned to each keypoint based on the local image gradients.
5. **Descriptor generation:** A numerical descriptor is created around each keypoint to represent the local image region.

Because SIFT considers different scales and assigns orientations to keypoints, it provides strong robustness to **scale and rotation changes**. SIFT descriptors can therefore be used for image matching and object recognition.

### Q3. What are keypoints and feature descriptors in image analysis?

#### Answer:

**Keypoints** are distinctive or important locations in an image that contain useful visual information. They are usually detected around areas such as **corners, edges, textures, and other distinctive structures**.

A **feature descriptor** is a numerical representation of the local image region surrounding a keypoint. It describes the characteristics of that region so that the same or similar features can be identified in another image.

For example, in SIFT, each detected keypoint is represented by a **128-dimensional feature descriptor**.

Therefore:

**Keypoint → Location of an important feature**

**Descriptor → Numerical representation of that feature**

Keypoints and descriptors are widely used in image matching, object recognition, tracking, and other computer vision applications.

#### Q4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

##### Answer:

HOG stands for **Histogram of Oriented Gradients**. It is a feature descriptor that represents an image by analyzing the **direction and magnitude of intensity gradients**.

In HOG, the image is divided into small regions called **cells**. The gradients within each cell are calculated and grouped into orientation histograms. Several cells are combined into larger **blocks**, and normalization is performed to improve robustness.

The main steps are:

1. Calculate image gradients.
2. Determine gradient magnitude and orientation.
3. Divide the image into cells.
4. Create orientation histograms for the cells.
5. Group cells into blocks.
6. Normalize the blocks.
7. Form the final HOG feature vector.

HOG is significant because it captures **shape, edges, and object structure** effectively. It is particularly useful in object detection and recognition tasks.

#### Q5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

##### Answer:

| Parameter                | SIFT                                         | HOG                                                          |
|--------------------------|----------------------------------------------|--------------------------------------------------------------|
| Full form                | Scale-Invariant Feature Transform            | Histogram of Oriented Gradients                              |
| Feature type             | Local feature descriptor                     | Gradient/shape descriptor                                    |
| Main information         | Local texture and structure                  | Shape and edge information                                   |
| Scale robustness         | Strong                                       | Not inherently scale invariant                               |
| Rotation robustness      | Strong                                       | Not inherently rotation invariant                            |
| Feature representation   | Keypoints and local descriptors              | Gradient orientation histograms                              |
| Computational complexity | Generally more computationally involved      | Generally simpler                                            |
| Image matching           | Highly suitable                              | Less suitable for direct local matching                      |
| Object recognition       | Suitable                                     | Suitable                                                     |
| Object detection         | Suitable                                     | Highly useful                                                |
| Typical applications     | Image matching, object recognition, tracking | Object detection, shape representation, pedestrian detection |

**Conclusion:** SIFT is more suitable when the task requires robust local feature matching under changes in scale and rotation. HOG is particularly useful when the main information of interest is the **shape and gradient structure** of an object.

#### Q6. Why is SIFT considered invariant to scale and rotation?

##### Answer:

SIFT is considered scale invariant because it detects keypoints across multiple image scales using a **scale-space representation** and Difference of Gaussian technique. Therefore, important features can be detected even when the size of an object changes.

SIFT is considered rotation invariant because an **orientation is assigned to each keypoint** based on the local gradient information. The feature descriptor is then constructed relative to this orientation.

As a result, when the same object is rotated or resized, its important local features can still be detected and matched.

Therefore, SIFT provides strong robustness against:

- Changes in scale
- Changes in rotation

#### Q7. Mention three real-world applications where HOG descriptors are commonly used.

##### Answer:

Three important real-world applications of HOG descriptors are:

1. **Pedestrian Detection:** HOG can represent the shape and edge structure of people and is commonly used in pedestrian detection systems.
2. **Object Detection:** HOG can be used to detect objects based on their shape and gradient patterns.
3. **Human/Shape Recognition:** HOG features can represent the structure and contours of human figures and other objects for recognition tasks.

Thus, HOG is particularly useful in applications where **shape and edge information** are important.

---

## Q8. Why is feature extraction performed before image classification or object detection?

### Answer:

Feature extraction is performed before image classification or object detection because raw images contain a very large amount of pixel-level information. Much of this information may not be directly useful for identifying an object.

Feature extraction converts the raw image into a more meaningful representation containing important characteristics such as:

- Edges
- Corners
- Shapes
- Textures
- Local patterns
- Gradient information

These extracted features make it easier for a classification or detection algorithm to distinguish between different objects.

Therefore, the general process is:

**Raw Image → Feature Extraction → Feature Representation → Classification/Detection**

Feature extraction can reduce the complexity of the input and provide useful information for subsequent computer vision tasks.

---

## Q9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

### Answer:

Handcrafted feature descriptors such as SIFT and HOG are designed using predefined mathematical techniques.

#### Advantages of handcrafted features:

1. They are based on well-understood mathematical principles.
2. They can work effectively with relatively small datasets.
3. Their features are generally interpretable.
4. They can be computationally efficient for specific tasks.
5. SIFT provides strong robustness to scale and rotation changes.

#### Limitations of handcrafted features:

1. Their performance depends heavily on manually selected algorithms and parameters.
2. They may not capture complex visual patterns as effectively as deep learning methods.
3. They can perform poorly when images have large variations in appearance.
4. They generally require separate feature engineering for different tasks.

#### Deep learning-based feature extraction:

Deep learning models, particularly **Convolutional Neural Networks (CNNs)**, can automatically learn useful features directly from training data. They can learn increasingly complex patterns from simple edges to higher-level object structures.

However, deep learning generally requires more computational resources and often requires large amounts of training data.

Therefore, handcrafted descriptors are useful for specific and well-defined computer vision tasks, while deep learning is generally more flexible for complex large-scale visual recognition problems.

---

## Q10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

### Answer:

Feature extraction techniques convert images into meaningful numerical representations that can be compared or processed by computer vision algorithms.

#### 1. Image Matching

Features such as SIFT keypoints and descriptors can be extracted from two images. Corresponding descriptors can then be matched to determine whether the images contain similar objects or scenes.

**Image 1 → Features**

**Image 2 → Features**

**Feature Matching → Similarity****2. Face Recognition**

Feature extraction can identify distinctive characteristics of a face, such as edges, shapes, and local patterns. These features can be compared with stored facial feature representations to recognize or verify a person.

**3. Object Detection**

Feature descriptors such as HOG can capture the shape and gradient structure of objects. These features can then be used by detection algorithms to identify objects within images.

Therefore, feature extraction acts as an important intermediate step between **raw image data and computer vision tasks such as matching, recognition, and detection.**